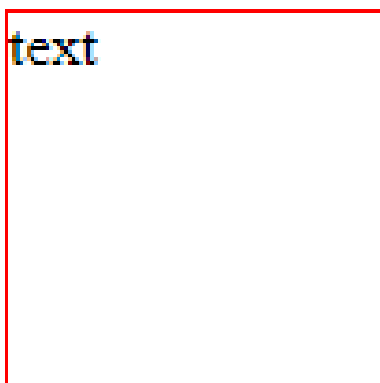


1 Работа со свойством display

Вы уже знаете, что все теги относятся к определенной блочной модели. При желании, однако, можно сменить модель с помощью свойства `display`. С помощью значения `block` данного свойства элемент можно сделать блочным, а с помощью значения `inline` - строчным.

Давайте сделаем так, чтобы теги `span` вели себя как блочные элементы:

```
<span>text</span>span {  
  display: block;  
  width: 100px;  
  height: 100px;  
  border: 1px solid red;  
  margin-bottom: 20px;  
}
```



А теперь сделаем так, чтобы теги `div` вели себя как строчные:

```
<div>text</div>  
<div>text</div>  
<div>text</div>div {  
  display: inline;  
  width: 100px;  
  height: 100px;  
  border: 1px solid red;  
}
```

Результат выполнения кода:

text text text

Задача 1

Пусть у вас в HTML коде есть несколько абзацев. Сделайте так, чтобы они вели себя как строчные элементы.

Задача 2

Пусть у вас в HTML коде есть несколько ссылок. Сделайте так, чтобы они вели себя как блочные элементы. Задайте им границу, ширину и высоту.

2 Особенности строчно-блочных элементов в CSS

В данном уроке мы с вами рассмотрим теги, которые при отображении на экране являются *строчно-блочных*. Строчно-блочные элементы сочетают особенности блочных и строчных элементов.

Вы пока знаете только один такой тег: `img`. Картинки, однако, не раскрывают все особенности этой модели, поэтому работу таких элементов посмотрим на примере тегов `span`, сменив им блочную модель, задав свойству `display` значение `inline-block`.

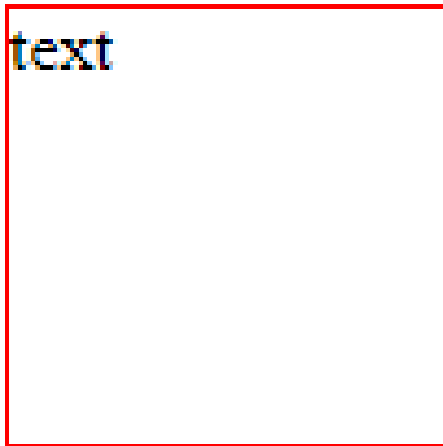
Ширина и высота строчно-блочного элемента

Строчно-блочного элементу можно задать ширину и высоту, подобно блочным элементам:

```
<span>text</span>span {  
  width: 100px;  
  height: 100px;
```

```
border: 1px solid red;  
display: inline-block;  
}
```

Результат выполнения кода:



Ширина строчно-блочного элемента

Если у строчно-блочного элемента не задана ширина, то он его ширина сформируется содержимым, подобно строчным элементам:

```
<span>text</span>span {  
  height: 100px;  
  border: 1px solid red;  
  display: inline-block;  
}
```

Результат выполнения кода:

text

Высота строчно-блочного элемента

Если у строчно-блочного элемента не задана высота, то его высота сформируется содержимым, подобно строчным элементам:

```
<span>text</span>span {  
  width: 100px;  
  border: 1px solid red;  
  display: inline-block;  
}
```

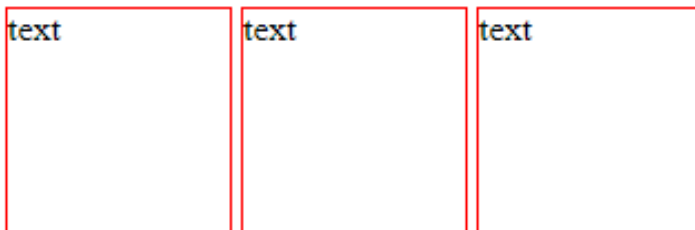
Результат выполнения кода:

text

Несколько строчно-блочного элементов рядом

Если расположить несколько строчно-блочного элементов рядом, то они выстроятся в ряд, подобно строчным элементам:

```
<span>text</span>
<span>text</span>
<span>text</span>span {
  width: 100px;
  height: 100px;
  margin-bottom: 20px;
  border: 1px solid red;
  display: inline-block;
}
```



Практические задачи

Задача 1

Даны дивы:

```
<div class="elem">text</div>
<div class="elem">text</div>
<div class="elem">text</div>div {
  width: 100px;
  height: 100px;
  border: 1px solid red;
}
```

Выстройте эти дивы в ряд, преобразовав их в строчно-блочные элементы.

3 Выравнивание строчных элементов

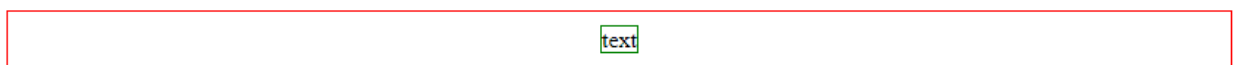
В предыдущем уроке мы с вами научились выравнивать блочные элементы. Описанный способ работает только для блочных элементов. Строчные элементы выравниваются по-другому - просто родителю следует задать свойство `text-align` в нужном значении.

Центрирование элементов

Для примера давайте поставим строчный элемент по центру:

```
<div class="parent">
  <span class="child">text</span>
</div>
.parent {
  text-align: center;
  padding: 10px 0;
  border: 1px solid red;
}
.child {
  border: 1px solid green;
}
```

Результат выполнения кода:



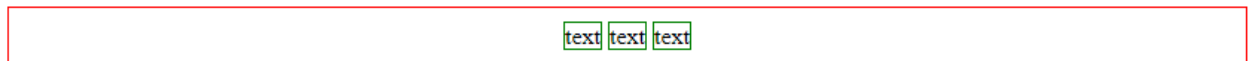
Несколько элементов

Это без проблем работает, если строчных элементов не один, а несколько:

```
<div class="parent">
  <span class="child">text</span>
  <span class="child">text</span>
  <span class="child">text</span>
</div>
```

```
</div> .parent {
  text-align: center;
  padding: 10px 0;
  border: 1px solid red;
}
.child {
  border: 1px solid green;
}
```

Результат выполнения кода:



Выравнивание по правому краю

А теперь давайте выровняем элементы по правому краю:

```
<div class="parent">
  <span class="child">text</span>
</div> .parent {
  text-align: right;
  padding: 10px 0;
  border: 1px solid red;
}
.child {
  border: 1px solid green;
}
```

Результат выполнения кода:



Практические задачи

Задача 1

Дан некоторый див, а в нем ссылки:

```
<div class="parent">
  <a href="">ссылка 1</a>
  <a href="">ссылка 2</a>
  <a href="">ссылка 3</a>
</div>.parent {
  padding: 10px 0;
  border: 1px solid red;
}
```

Поставьте ссылки по центру этого дива.

Задача 2

Дан некоторый див, а в нем ссылка:

```
<div class="parent">
  <a href="">ссылка</a>
</div>.parent {
  padding: 10px 0;
  border: 1px solid red;
}
```

Поставьте эту ссылку по правому краю.

Задача 3

Модифицируйте предыдущую задачу так, чтобы ссылка не прижималась плотно к правому краю. Для этого родительскому элементу добавьте правый padding.

4 Выравнивание строчно-блочных элементов

Строчно-блочные элементы выравниваются так же, как и строчные - с помощью `text-align` в нужном значении, заданном для родителя.

Центрирование элементов

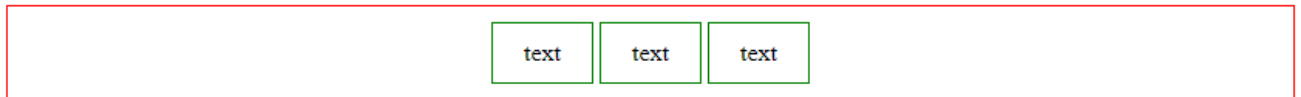
Для примера давайте поставим строчно-блочные элементы по центру:

```
<div class="parent">
  <span class="child">text</span>
```



```
<span class="child">text</span>
<span class="child">text</span>
</div>.parent {
  text-align: center;
  padding: 10px 0;
  border: 1px solid red;
}
.child {
  display: inline-block;
  padding: 10px 20px;
  border: 1px solid green;
}
```

Результат выполнения кода:



Выравнивание по правому краю

Выровняем элемент по правому краю:

```
<div class="parent">
  <span class="child">text</span>
</div>.parent {
  text-align: right;
  padding: 10px 0;
  border: 1px solid red;
}
.child {
  display: inline-block;
  padding: 10px 20px;
  border: 1px solid green;
}
```

Результат выполнения кода:

Практические задачи

Задача 1

Даны дивы внутри некоторого родителя:

```
<div class="parent">
  <div class="child">text 1</div>
  <div class="child">text 2</div>
  <div class="child">text 3</div>
</div>.parent {
  padding: 10px 0;
  border: 1px solid red;
}
.child {
  padding: 10px;
  border: 1px solid green;
}
```

Преобразуйте дивы-потомки в строчно-блочные элементы, задайте им горизонтальный `margin` в 5px, а затем отцентрируйте их по горизонтали относительно своего родителя.

Задача 2

Дан див внутри некоторого родителя:

```
<div class="parent">
  <div class="child">text</div>
</div>.parent {
  padding: 10px 0;
  border: 1px solid red;
}
.child {
  padding: 10px;
```

```
border: 1px solid green;  
}
```

Преобразуйте див-потомок в строчно-блочный элемент, затем выровняйте его по правому краю родителя.

Задача 3

Модифицируйте предыдущую задачу так, чтобы див не прижимался плотно к правому краю. Для этого родительскому элементу добавьте правый `padding`.

Задача 4

Модифицируйте предыдущую задачу так, чтобы див не прижимался плотно к правому краю. Для этого элементу-потомку поставьте правый `margin`.

Задача 5

Дана ссылка внутри некоторого родителя:

```
<div class="parent">  
  <a href="" class="child">text</a>  
</div>.parent {  
  padding: 10px 0;  
  border: 1px solid red;  
}  
.child {  
  width: 100px;  
  padding: 10px;  
  border: 1px solid green;  
}
```

Преобразуйте ссылку в строчно-блочный элемент, затем выровняйте его по правому краю родителя.

Задача 6

Модифицируйте предыдущую задачу так, чтобы текст внутри ссылки был выровнен по центру этой ссылки.

5 Флекс элементы на CSS

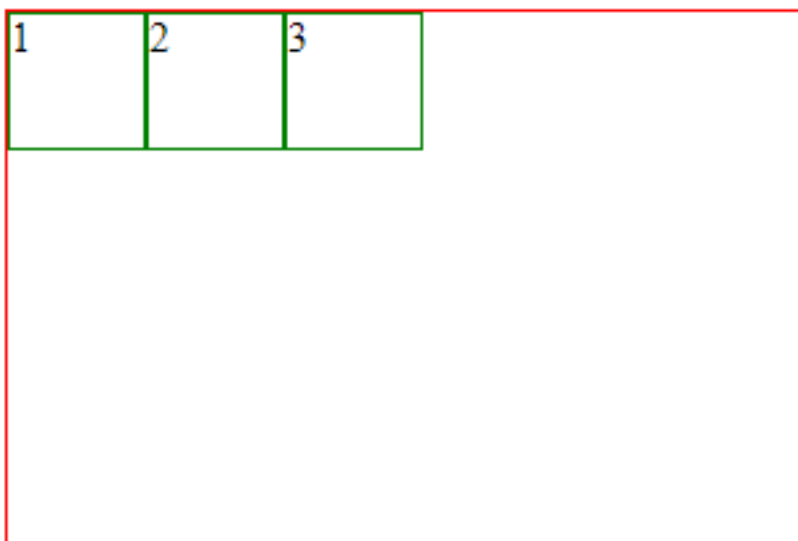
Кроме изученных нами типов блочных моделей есть еще одна - флекс элементы. Для того, чтобы получить такие элементы, нужно *родителю* этих

элементов написать свойство `display` в значении `flex`. Сам родитель останется блочным элементом, а вот его потомки станут флекс элементами. Флекс элементы, подобно блочным могут иметь ширину и высоту, `margin` и `padding`. Однако, в отличие от блочных, по умолчанию флекс элементы выстраиваются в ряд внутри своего родителя.

Давайте попробуем:

```
<div class="parent">
  <div class="child">1</div>
  <div class="child">2</div>
  <div class="child">3</div>
</div>
.parent {
  display: flex; /* потомки будут флекс элементами */
  width: 300px;
  height: 200px;
  border: 1px solid red;
}
.child {
  width: 50px;
  height: 50px;
  border: 1px solid green;
}
```

Результат выполнения кода:

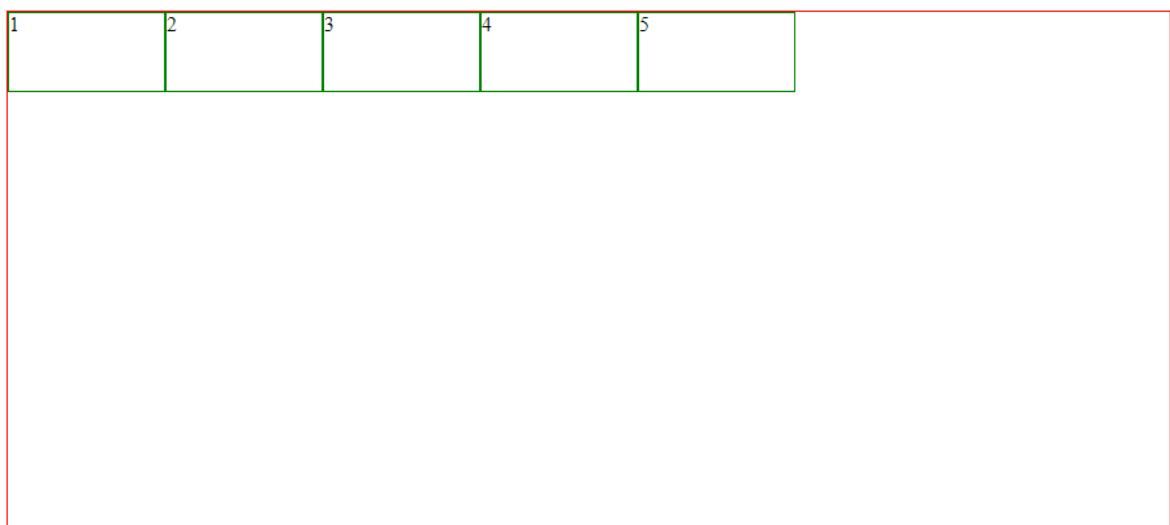


Задача 1

Повторите пример используя `display:flex` для 5 элементов зеленой рамки

Дам подсказку(создайте `div` с классом `parent` и в нем 5 `div` элементов с классом `child` и получится такой результат

```
.parent {  
    display: flex;  
    width: 900px;  
    height: 400px;  
    margin: 20px auto;  
    border: 1px solid red;  
}  
  
.child {  
    width: 120px;  
    height: 60px;  
    border: 1px solid green;  
}
```



Высота родителя

Если родителю флекс элементов не задана высота, то по высоте он будет растягиваться своим содержимым, то есть высотой потомков:

```
<div class="parent">
  <div class="child">1</div>
  <div class="child">2</div>
  <div class="child">3</div>
</div>.parent {
  display: flex;
  width: 300px;
  border: 1px solid red;
}
.child {
  width: 50px;
  height: 50px;
  border: 1px solid green;
}
```

Результат выполнения кода:

Ширина родителя

Если родителю флекс элементов не задана ширина, то по ширине он займет все доступное место:

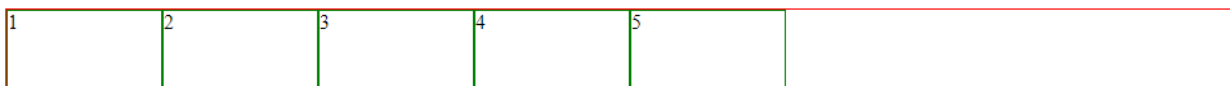
```
<div class="parent">
  <div class="child">1</div>
  <div class="child">2</div>
  <div class="child">3</div>
</div>.parent {
  display: flex;
  border: 1px solid red;
}
.child {
  width: 50px;
  height: 50px;
  border: 1px solid green;
}
```

Результат выполнения кода:



Задача 2

По примеру выше повторите задачу, только нужно полностью растянуть блок имеющий красную рамку.



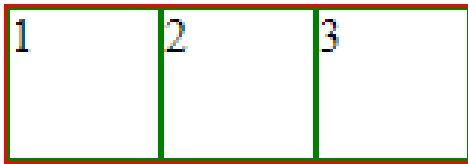
Строчно-блочный родитель

Вместо значения flex для свойства display родителю можно поставить значение inline-flex. В этом случае этот родитель будет вести себя, как строчно-блочный элемент.

Например, если не задать ширину родителя, то его ширина сформируется шириной потомков:

```
<div class="parent">
  <div class="child">1</div>
  <div class="child">2</div>
  <div class="child">3</div>
</div>
.parent {
  display: inline-flex;
  border: 1px solid red;
}
.child {
  width: 50px;
  height: 50px;
  border: 1px solid green;
}
```

Результат выполнения кода:

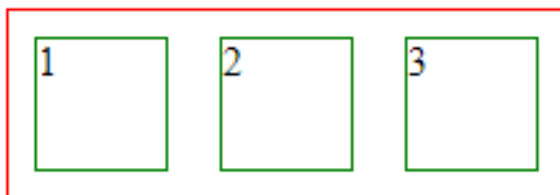


Отступы margin потомкам

Флекс элементам можно задать отступы margin:

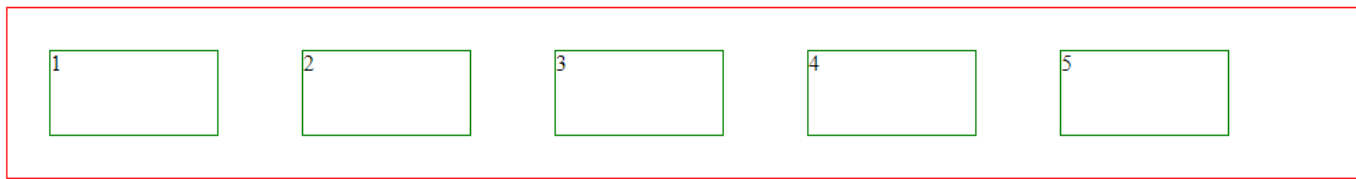
```
<div class="parent">
  <div class="child">1</div>
  <div class="child">2</div>
  <div class="child">3</div>
</div>.parent {
  display: inline-flex;
  border: 1px solid red;
}
.child {
  margin: 10px;
  width: 50px;
  height: 50px;
  border: 1px solid green;
}
```

Результат выполнения кода:



Задача 3

Дополните предыдущий пример сделав красный блок на всю ширину и увеличьте сами элементы(которые с зеленой рамкой и отступы между ними)



6 Выравнивание flex элементов

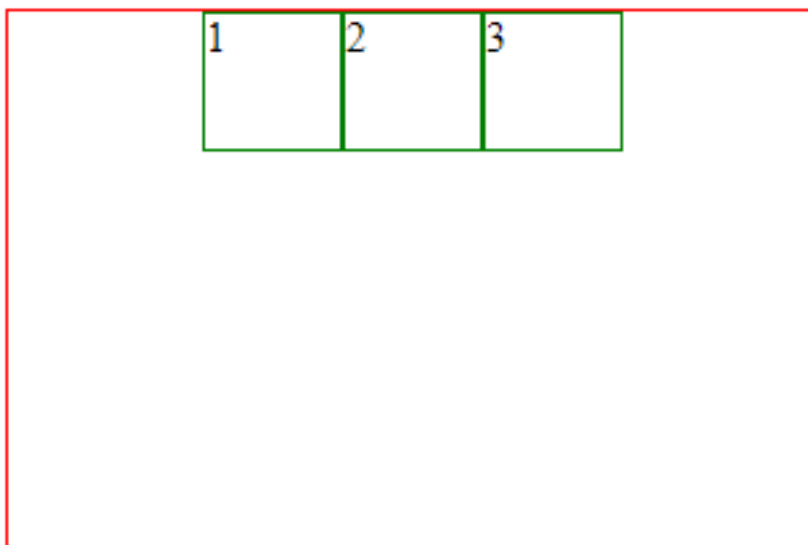
Для выравнивания флекс элементов используется специальное свойство `justify-content`. Оно принимает достаточно много значений, которые мы и изучим в данном уроке.

Значение center

Значение `center` позволяет выровнять элементы по центру:

```
.parent {  
  display: flex;  
  justify-content: center;  
}
```

Результат выполнения кода:



Задача 1

Повторите страницу по данному образцу:

1	2	3	4	5
---	---	---	---	---

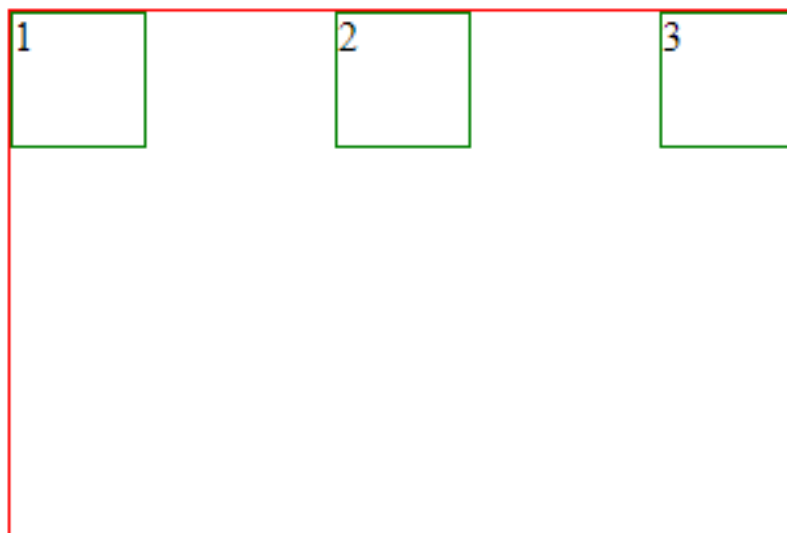
Значение space-between

Значение `space-between` позволяет сделать так, чтобы элементы равномерно распределились по своему родителю, а расстояние между ними рассчиталось автоматически. При этом первый элемент будет прижат к левому краю, а последний - к правому.

Давайте посмотрим на примере:

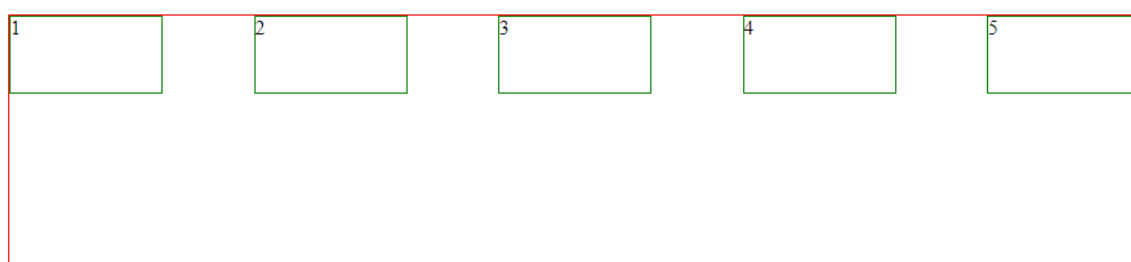
```
.parent {  
  display: flex;  
  justify-content: space-between;  
}
```

Результат выполнения кода:



Задача 2

Повторите страницу по данному образцу:



Значение space-around

Значение `space-around` работает похожим образом, только делает так, чтобы первый и последний элемент отступали от края родителя.

С первого взгляда может показаться, что расстояние между элементами и между крайними элементами и родителем будут равны, однако, это не так - расстояние между элементами будет в 2 раза больше, чем между родителем и крайним элементом.

Посмотрим на примере:

```
.parent {  
  display: flex;  
  justify-content: space-around;  
}
```

Результат выполнения кода:

1	2	3
---	---	---

Задача 3

Повторите страницу по данному образцу:

1	2	3	4	5
---	---	---	---	---